Reporte

Nombre: Equipo 6 Luis David Cigala Montes Jared Alejandro Garcia Alcocer Gabriel Sánchez Rivera	**Matrícula**: 2120993 2119041 2121131
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Actividad 4, Reflexión de errores (TechStore)
Fecha: 10/28/26	
Bibliografia (Formato APA): Archivo de la actividad 4 brindada por el ingeniero	

Objetivo:

Documentar y reflexionar sobre tres errores de implementación del núcleo de negocio de TechStore (unidades, Domain Events y umbral de descuento): cómo se manifiestan, cómo se detectan, cómo se corrigen y qué se aprende de cada uno.

Procedimiento:

Cómo se documentaron los errores

Para documentar los errores se trabajó sobre una copia del proyecto TechStore. En cada caso se introdujo el error en el código, se ejecutaron las 26 pruebas de dominio y de escenarios (test_domain.py y test_escenarios.py) junto con los scripts probar_*.py, y se registró qué falló. Con el código original, las 26 pruebas pasan sin fallos y python cargar_csv.py carga los 16 pedidos en DRAFT.

Error 1: contar renglones en lugar de unidades

Qué pasó. La propiedad units de Order se escribió como el número de renglones del pedido en vez de la suma de cantidades:

```
# Incorrecto
return len(self.items)

# Correcto
return sum(item.quantity for item in self.items)
```

Cómo se detectó. Al ejecutar python cargar_csv.py los pedidos mostraron unidades equivocadas: O001 aparecía con 2 unidades (debían ser 4), O008 con 1 (debían ser 20) y O009 con 1 (debían ser 19). Además fallaron 7 pruebas, entre ellas test_pedido_o001_recibe_descuento_del_10_por_ciento, test_E07_se_permiten_20_unidades, test_E08_no_se_permiten_mas_de_20_unidades y test_E10_tres_unidades_no_cumplen_minimo.

Por qué ocurre. El límite de 20 y la regla del 10% (mínimo 4 unidades) dependen de unidades, no de líneas. Con el error, un pedido de 20 piezas en un solo renglón contaría como 1 y el límite nunca se activaría; en cambio, O001 (2 renglones, 4 unidades) dejaría de cumplir la regla de las 4 unidades.

Cómo se solucionó. Se restauró units para que sume item.quantity de cada renglón. Con eso las 26 pruebas volvieron a pasar y O001 volvió a mostrar 4 unidades y 2100.00 MXN.

Qué se aprendió. Un nombre de propiedad ambiguo es fuente de errores: conviene distinguir en el vocabulario “renglones” de “unidades” y probar pedidos donde ambos valores sean distintos. Las pruebas basadas en los datos del paquete (O001, O008, O009) sirvieron justo para eso.

Error 2: registrar el Domain Event antes de validar

Qué pasó. En ship() se escribió la línea que agrega el evento OrderShipped al inicio del método, antes de las validaciones, y se olvidó quitar la que ya estaba al final:

```
def ship(self, tracking_number):
    self.events.append(DomainEvent("OrderShipped", self.id))  # antes de
validar
    if self.status != OrderStatus.CONFIRMED:
        raise InvalidOrderTransition(...)
    ...
```

Cómo se detectó. Fallaron 4 pruebas: test_no_se_puede_enviar_sin_guia, test_E11_no_se_envia_sin_guia, test_E14_pedido_termina_en_returned y test_E15_pedido_termina_en_delivered. En los casos fallidos (envío sin guía) la lista de eventos quedaba con un OrderShipped de algo que nunca ocurrió; en los casos exitosos el evento quedaba duplicado.

Por qué ocurre. Un Domain Event afirma que un hecho ya sucedió. Si se registra antes de validar, inventario, correo o facturación podrían reaccionar a un envío que fue rechazado. La regla del proyecto es validar primero, cambiar el estado después y registrar el evento al final.

Cómo se solucionó. Se dejó una sola línea de evento, al final de ship(), después de cambiar el estado, igual que en confirm(), cancel(), deliver() y return_order(). Las pruebas volvieron a pasar.

Qué se aprendió. Las pruebas no deben revisar solo el estado final, sino también la lista de eventos tras una operación inválida (estado sin cambio y events vacío o sin eventos nuevos). Esa verificación fue la que expuso el error.

Error 3: usar >= en lugar de > en el umbral de 500 MXN

Qué pasó. En MinimumTotalSpecification la comparación se cambió de “mayor que” a “mayor o igual que”:

```
# Incorrecto
return order.total.amount >= self.minimum

# Correcto
return order.total.amount > self.minimum
```

Cómo se detectó. Este fue el error más difícil de ver: las 26 pruebas siguieron pasando y probar_descuentos.py seguía mostrando 0.00% para O010. Solo al evaluar directamente MinimumTotalSpecification(500) sobre O010 (total exactamente 500.00) apareció la diferencia: devolvía True con >= y False con >.

Por qué pasó desapercibido. O010 tampoco cumple las demás condiciones del 10% (solo tiene 2 unidades), así que el descuento final seguía siendo 0% y el error quedaba escondido. El escenario E09 verifica el descuento completo, no la frontera de la Specification por separado.

Cómo se solucionó. Se restauró el operador > (la guía exige que el umbral sea estricto) y se propone agregar una prueba directa de la frontera: que MinimumTotalSpecification(500) sea falsa para un total de 500.00 y verdadera para 500.01.

Qué se aprendió. Que todas las pruebas pasen no demuestra que no haya errores. Los valores frontera (500, 5000, 20 unidades) deben probarse a nivel de cada Specification, y no solo a través del resultado final, porque las demás reglas pueden ocultar el fallo.

Resumen

Error	Pruebas que fallaron	Cómo se detectó	Corrección
Contar renglones en lugar de unidades	7 de 26	cargar_csv.py y pruebas E01, E07, E08, E10	units suma item.quantity
Evento antes de validar	4 de 26	Pruebas E11, E14, E15 y test de envío sin guía	Evento al final, después del cambio de estado
>= en lugar de > en 500 MXN	0 de 26	Solo al evaluar la Specification con total 500.00	Operador > y prueba de frontera

Reflexión general

Los tres errores comparten una causa: la regla de negocio estaba bien pensada, pero se implementó con un detalle pequeño que la cambiaba (una propiedad mal definida, un evento en el lugar equivocado, un operador de comparación). Dos de ellos fueron

detectados por las pruebas porque comparan resultados concretos de los datos del paquete; el tercero mostró que una suite que pasa no garantiza que el código sea correcto si no se prueban los valores frontera de forma directa.

También se confirmó la ventaja de tener las reglas en el dominio y no en la API: para corregir cada error bastó con modificar una sola línea en domain.py o specifications.py, sin tocar api.py ni los endpoints, y todas las pruebas permitieron comprobar la corrección de inmediato.

Conclusiones

Validar antes de cambiar el estado, registrar eventos solo al final, contar unidades y no renglones, y probar los umbrales exactos son prácticas que evitan que el sistema acepte operaciones inválidas o anuncie hechos que no ocurrieron. Documentar los errores permitió entender no solo qué se corrigió, sino por qué la regla del negocio exige ese comportamiento.